Data-Oriented Design vs. Object-Oriented Structures in High-Performance Systems Programming

Spyridon Arvanitis it2023003@hua.gr
Dimitrios Xiromeritis it2023052@hua.gr
Alexios Politis it2023100@hua.gr

Harokopio University of Athens, Greece
Course: Systems Programming
June 21, 2026

Abstract

This study investigates processing bottlenecks caused by the "Memory Wall" and evaluates data layout paradigms to minimize memory stall cycles. Using a C11/Raylib-based Particle System simulation, we compare Object-Oriented Programming (OOP) layouts with Data-Oriented Design (DOD) principles. We evaluate the performance of Array of Structures (AoS) versus Structure of Arrays (SoA) across workloads of up to 2,000,000 particles. Additionally, we implement Single Instruction, Multiple Data (SIMD) vector pipelines using Intel AVX/AVX2 intrinsics with strict 64-byte memory alignment. Hardware micro-clocks, Valgrind's `Cachegrind`, and `gprof` were used to measure performance metrics. The results show that optimizing data layouts for spatial contiguity reduces data cache line bloat and L1/Last-Level (LL) cache invalidations, significantly improving instruction throughput.

1 Introduction

The performance gap between CPU execution speeds and DRAM access latencies—known as the Memory Wall—remains a primary bottleneck in high-performance computing. To combat this latency mismatch, modern microprocessors rely on hierarchical caching topologies consisting of L1, L2, and shared L3 static random-access memory (SRAM) banks [5]. Whenever a core executes an operation requiring data from main memory, it evaluates the fast cache layers first. If the requested data is not present, a "cache miss" occurs, halting pipeline execution while the data is fetched from DRAM.

To maximize throughput, the CPU memory controller reads data in 64-byte segments known as *cache lines* [5]. Hardware *spatial prefetchers* also analyze execution streams to predict and load sequential memory addresses into the cache proactively [5].

Despite these hardware mechanisms, software design patterns often degrade cache efficiency. Traditional Object-Oriented Programming (OOP) groups conceptual entities, bundling data fields and behaviors within classes or structs. While intuitive for developers, this approach reduces execution pipeline speed [6] when iterating over large arrays if the algorithm only accesses a small subset of the fields per cycle.

This paper compares conventional OOP memory arrangements with Data-Oriented Design (DOD). By restructuring data to prioritize hardware consumption over abstract entities [6], we show how developers can reduce cache pollution, utilize prefetchers effectively, and maximize vector pipeline utilization.

2 Microarchitectural Mechanics & Memory Paradigms

This benchmark platform compares two fundamental data allocation models: the Array of Structures (AoS) for encapsulation, and the Structure of Arrays (SoA) for composition.

2.1 The Array of Structures (AoS) Pattern

The AoS paradigm groups data by entity. Each element is an isolated instance containing its component properties packed within a single struct:

```
typedef struct {
    Vector2 pos;            // 8 bytes
    Vector2 vel;            // 8 bytes
    Vector2 acceleration;   // 8 bytes
    Color startColor;       // 4 bytes
    Color endColor;         // 4 bytes
    float startSize;        // 4 bytes
    float endSize;          // 4 bytes
    float rotation;         // 4 bytes
    float angularVelocity;  // 4 bytes
    float lifeTime;         // 4 bytes
    float age;              // 4 bytes
    float mass;             // 4 bytes
    float drag;             // 4 bytes
} ParticleAoS;              // Total Size = 64
    bytes
```

Listing 1: AoS Entity Data Packaging Configuration

In this setup, if a routine updates positions, the loop only requires the spatial fields `pos` and `vel`, totaling 16 bytes of data. However, when the processor reads `particles[i].pos`, it fetches an entire 64-byte cache line. Consequently, the remaining 48 bytes (`startColor`, `lifeTime`, `drag`, etc.) are loaded into the L1 cache as unused data.

The memory access stride length S equals the object width:

$$S = \text{sizeof(ParticleAoS)} = 64 \text{ bytes} \quad (1)$$

This stride prevents multiple particles from fitting within a single L1 cache line, causing cache pollution.

2.2 Architectural Isolation & Dynamic Dispatch

Pure ISO C11 lacks formal object-oriented primitives like virtual method tables (vtables). In production OOP software (e.g., C++ or C#), an object loop often triggers updates via virtual functions (e.g., `particle->update()`).

For this experiment, dynamic dispatch and internal function pointers were omitted from `ParticleAoS`. This isolates the **Data Cache (D-Cache)**. Including function pointers would have shifted the bottleneck to the **Instruction Cache (I-Cache)**, causing I-cache evictions and branch mispredictions.

The AoS baseline was also allocated as a contiguous memory block via `malloc` rather than an array of scattered heap pointers, providing a best-case scenario without heap fragmentation. This isolates performance degradation strictly to cache line bloat.

2.3 The Structure of Arrays (SoA) Pattern

Data-Oriented Design (DOD) organizes data around transformations rather than entities [4]. Instead of an array of structs, components are separated into parallel arrays:

```
typedef struct {
    // Data used in the update loop
    float* posX;
    float* posY;
    float* velX;
    float* velY;
    // Data not used in the update loop
    float* accX;
    float* accY;
    Color* startColor;
    Color* endColor;
    float* startSize;
    float* endSize;
    float* rotation;
    float* angularVelocity;
    float* lifeTime;
    float* age;
    float* mass;
    float* drag;
    void* memoryBlock;
} ParticleSystemSoA;
```

Listing 2: SoA Contiguous Component Vector Fields

A single memory block is partitioned into aligned segments. During a position update, the CPU requests sequential indices across the `posX`, `posY`, `velX`, and `velY` arrays.

Since every value in these arrays is used in the computation, the memory access stride is minimized:

$$S = \text{sizeof(float)} = 4 \text{ bytes} \quad (2)$$

A 64-byte cache line now holds 16 consecutive floating-point values. This improves data cache efficiency and allows hardware prefetchers to load upcoming data accurately.

3 System Optimization & Explicit SIMD Vectorization

To leverage the SoA layout further, the pipeline uses explicit loop vectorization via Intel AVX/AVX2 intrinsics.

3.1 Memory Alignment Constraints

SIMD execution engines require base address alignment to avoid access penalties. Unaligned loads (`_mm256_loadu_ps`) across cache line boundaries trigger split memory bus requests.

The SoA memory manager computes 64-byte alignment offsets to prevent this:

```
#define ALIGN64(size) (((size) + 63) & ~63)
size_t floatArraySize = ALIGN64(count * sizeof(
    float));
```

This aligns the base index of each array to a physical cache line boundary, allowing the use of aligned memory load and store instructions (`_mm256_load_ps` / `_mm256_store_ps`) to reduce latency.

3.2 Branchless Vectorized Physics Pipelining

Physics loops typically use conditional branches (`if` statements) for boundaries. With large, random datasets, the CPU's Branch Prediction Unit (BPU) often mispredicts these conditions, causing pipeline flushes [7].

The SIMD implementation removes branches using bitwise mask operations and vector blends. Eight particles are loaded, updated, tested for collisions, and stored concurrently without conditional jumps:

```
__m256 dt      = _mm256_set1_ps(delta);
__m256 gravity = _mm256_set1_ps(GRAVITY_Y *
    delta);
__m256 drag    = _mm256_set1_ps(DRAG);
__m256 zero    = _mm256_setzero_ps();
__m256 screenW = _mm256_set1_ps((float)
    SCREEN_WIDTH);
__m256 bounce  = _mm256_set1_ps(-
    BOUNCE_DAMPENING);

for (uint32_t i = 0; i < simdCount; i += 8) {
    __m256 px = _mm256_load_ps(&soa->posX[i]);
    __m256 py = _mm256_load_ps(&soa->posY[i]);
    __m256 vx = _mm256_load_ps(&soa->velX[i]);
    __m256 vy = _mm256_load_ps(&soa->velY[i]);

    vy = _mm256_add_ps(vy, gravity);
    vx = _mm256_mul_ps(vx, drag);
    vy = _mm256_mul_ps(vy, drag);

    px = _mm256_add_ps(px, _mm256_mul_ps(vx, dt)
    );
    py = _mm256_add_ps(py, _mm256_mul_ps(vy, dt)
    );

    __m256 maskL = _mm256_cmp_ps(px, zero,
    _CMP_LE_OQ);
    __m256 maskR = _mm256_cmp_ps(px, screenW,
    _CMP_GE_OQ);
    __m256 maskOOB = _mm256_or_ps(maskL, maskR);
```

```
    __m256 bVx = _mm256_mul_ps(vx, bounce);
    vx = _mm256_blendv_ps(vx, bVx, maskOOB);

    px = _mm256_max_ps(px, zero);
    px = _mm256_min_ps(px, screenW);

    _mm256_store_ps(&soa->posX[i], px);
    _mm256_store_ps(&soa->posY[i], py);
    _mm256_store_ps(&soa->velX[i], vx);
    _mm256_store_ps(&soa->velY[i], vy);
}
```

Listing 3: Vectorized Branchless Physics Pipeline

4 Experimental Methodology

The application includes an automated benchmarking routine. Scenarios use a fixed random seed (`SetRandomSeed(42)`) for repeatability and run for 10.0 seconds.

The benchmarks evaluate:

1. **Data Organization Paradigm:** Array of Structures (AoS) vs. Structure of Arrays (SoA Scalar) vs. Structure of Arrays (SoA SIMD Vectorized).
2. **Physics Matrix State:** Simple linear positional integration vs. full complex physics modeling (incorporating drag coefficients, gravity, and boundary collisions).
3. **Dataset Scaling:** 50,000, 500,000, and 2,000,000 particles to push memory usage beyond standard L2/L3 cache capacities.

4.1 Target Hardware Architecture

Benchmarks were run natively on an Intel Core i7-13620H processor with 48 KB of L1 Data Cache ($L1d$) per Performance-core.

Given the standard 64-byte cache line width, this provides the following L1 capacity:

$$\text{Capacity}_{L1d} = \frac{48 \times 1024 \text{ bytes}}{64 \text{ bytes/line}} = 768 \text{ cache lines} \quad (3)$$

A workload of 2,000,000 particles guarantees L1 cache saturation, testing the prefetching efficiency of the memory layouts.

4.2 CPU Profiling Isolation

Rendering overhead can skew CPU performance metrics. Since Raylib uses OpenGL, mixing data logging with display updates introduces noise from GPU pipeline stalls or VSync.

The benchmark isolates CPU physics calculations from graphical processing. Hardware timers (`GetTime()`) measure the update loops exclusively:

```
double startTime = GetTime();
    simState->update(simState, delta);
double endTime = GetTime();
simState->lastPhysicsLoopUpdate = endTime -
    startTime;
```

The visual rendering pipeline is excluded from the timing metrics. Cache misses are tracked using Valgrind's `Cachegrind` tool.

5 Empirical Analysis & Discussion

5.1 The Small-Struct Anomaly

Initial tests used a 24-byte structure for the `ParticleAoS` dataset. In these tests, AoS and SoA models showed similar processing times.

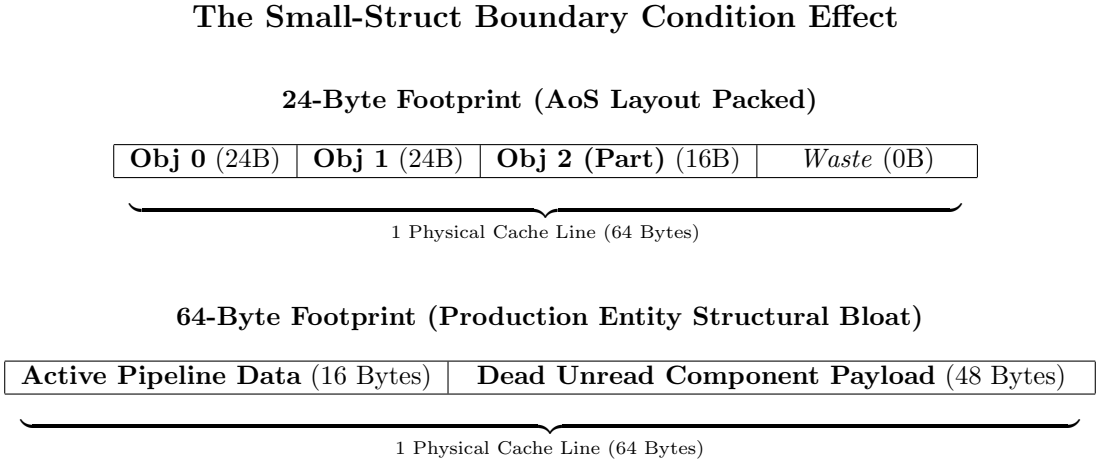

Figure 1: Memory layout allocation comparison demonstrating how standard 64-byte cache lines ingest alternative data payloads under different struct sizes.

This occurred because a 24-byte struct allows the memory controller to pack multiple entities into a single 64-byte cache line:

$$\text{Density}_{\text{CacheLine}} = \frac{64 \text{ bytes}}{24 \text{ bytes/entity}} \approx 2.66 \text{ entities} \quad (4)$$

Since subsequent iterations found data already in the L1 cache, this layout mimicked the dense sequential behavior of SoA.

To simulate a more realistic game entity, the struct was expanded to 64 bytes, including colors, lifetime coun-

ters, and rotation data. This highlights the cache bloat penalty in OOP layouts during sub-component updates.

5.2 Hardware Memory Trace Analysis

The Cachegrind data for 2,000,000 particles (Table 1) shows why the AoS layout is slower.

While the L1 miss rate remains near 2.5%, the absolute volume of transactions differs. Bloated with 48 bytes of unread data per entity, the AoS layout generates over 5 billion data memory references per frame array. This causes 123.5 million physical L1 cache misses, stalling the ALU.

The SoA layout reduces data references to 2.3 billion and L1 misses to 54.7 million. Contiguous data streams also allow the compiler to generate cleaner assembly, reducing Instruction Reads (Ir) from 13.5 billion to 6.2 billion. AVX SIMD lowers instruction reads to 3.0 billion by processing 8 operations per instruction.

5.3 Execution Latency

CPU timings (Table 2) correlate with the cache traces.

For 2,000,000 particles with complex physics, the AoS loop takes 9.0980 ms per frame. In a 16.6ms (60 FPS) budget, this physics update consumes 55% of the frame time.

The SoA Scalar implementation takes 4.3491 ms. Adding AVX SIMD and branchless logic lowers the execution time to 1.7431 ms. This results in a 5.2x speedup compared to the baseline AoS implementation, achieved solely by organizing data to optimize L1 cache utilization.

6 Conclusion

This research demonstrates that memory layout significantly impacts application performance. Traditional AoS layouts can introduce memory overhead and cache pollution during sequential batch processing [6].

By adopting a Data-Oriented Design (SoA) and utilizing SIMD pipelines, execution speeds can be improved on the same hardware. These results highlight the importance of hardware-aware data structures in performance-critical software.

References

[1] R. Sanches, *Raylib Architecture Framework for Visual Interactive Computing Environments*, v4.0, GitHub, `https://github.com/raysan5/raylib`.

[2] N. Nethercote and J. Seward, *Valgrind: A Framework for Heavyweight Dynamic Binary Analysis*, Proceedings of PLDI '07, 2007.

[3] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2026.

[4] R. Fabian, *Data-Oriented Design: Software Engineering for Limited Resources and Short Schedules*, 1st ed., Richard Fabian, 2018.

[5] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Pearson, 2015.

[6] M. Acton, "Data-Oriented Design and C++," *CppCon 2014*, Bellevue, WA, 2014. [Online]. Available: `https://youtu.be/rX0ItVEVjHc`.

[7] A. Fog, *Optimizing software in C++: An optimization guide for Windows, Linux and Mac platforms*, Copenhagen University College of Engineering, 2024. [Online]. Available: `https://www.agner.org/optimize/`.

Table 1: Instruction & D-Cache Profile Metrics (2,000,000 Particles, No Collisions)

Architecture Paradigm	**Instruction Reads (Ir)**	**Data References (Dr+Dw)**	**L1 Data Misses**	**L1 Miss Rat**
AoS (OOP)	13,542,772,090	5,011,810,484	123,570,599	2.5%
SoA (Scalar DOD)	6,263,079,858	2,316,322,807	54,709,755	2.4%
SoA (SIMD Vectorized)	3,028,685,289	1,149,481,573	29,422,834	2.6%

Table 2: Average Loop Execution Times (ms) Across Scaling Workloads and Complexities

Memory Layout	**Simple Physics (No Collisions)**			**Complex Physics (Gravity & Walls)**		
	50k Nodes	**500k Nodes**	**2M Nodes**	**50k Nodes**	**500k Nodes**	**2M Nodes**
AoS (Object-Oriented)	0.2332 ms	2.0487 ms	7.5553 ms	0.2547 ms	2.5001 ms	9.0980 ms
SoA (Scalar DOD)	0.0511 ms	0.5095 ms	1.6466 ms	0.1074 ms	1.0645 ms	4.3491 ms
SoA (SIMD Vectorized)	0.0427 ms	0.4157 ms	1.4326 ms	**0.0481 ms**	**0.4810 ms**	**1.7431 ms**